

Laboratorio 2: optimización no lineal con Python

Optimización y Análisis de Redes

Víctor Aceña Gil / Antonio Alonso Ayuso

2026-06-16

Table of contents

1	Introducción	2
2	Fundamentos teóricos y herramientas de Python	2
2.1	Algoritmo del descenso de gradiente manual	2
2.1.1	Criterio de parada	3
2.2	Optimización de caja negra con <code>scipy.optimize.minimize</code>	3
2.2.1	Optimización sin restricciones: el método BFGS	3
2.2.2	Optimización con restricciones: el método SLSQP	4
2.3	Optimización Convexa Disciplinada con <code>CVXPY</code>	5
2.3.1	Reglas DCP Básicas	5
2.3.2	Obtención de Soluciones Duales (Multiplicadores de Lagrange)	5
2.4	Comparativa: SciPy vs CVXPY	6
3	Ejercicios de laboratorio propuestos	6
3.1	Ejercicio 1: descenso de gradiente manual	6
3.1.1	Tareas:	7
3.2	Ejercicio 2: optimización sin restricciones con SciPy	7
3.2.1	Tareas:	8
3.3	Ejercicio 3: optimización restringida con CVXPY y SciPy (Teoría y Práctica)	8
3.3.1	Tareas teóricas:	8
3.3.2	Tareas de código:	9

1 Introducción

La optimización no lineal es una de las disciplinas matemáticas con mayor impacto en la práctica industrial, científica y tecnológica. Mientras que algunos problemas de juguete pueden resolverse de manera analítica (por ejemplo, igualando el gradiente a cero o aplicando las condiciones de Karush-Kuhn-Tucker), en la práctica la complejidad de las funciones y el número de variables nos obligan a recurrir a **algoritmos numéricos iterativos**.

El objetivo de este laboratorio es aprender a relacionar los fundamentos teóricos de la optimización matemática con el uso de herramientas profesionales en Python. A lo largo de la sesión aprenderemos a:

1. Diseñar e implementar desde cero el algoritmo clásico de **descenso de gradiente**, analizando la influencia de la tasa de aprendizaje y el comportamiento iterativo del algoritmo.
2. Utilizar la biblioteca científica estándar **scipy.optimize** (en concreto, su función central **minimize**), entendiendo cómo configurar solvers libres de restricciones (como **BFGS**) y solvers con restricciones generales (como **SLSQP**).
3. Utilizar la biblioteca especializada **CVXPY** para resolver problemas de optimización convexa disciplinada, explorando sus ventajas en cuanto a declaración matemática y obtención directa de las **soluciones duales (multiplicadores de Lagrange)**.

2 Fundamentos teóricos y herramientas de Python

2.1 Algoritmo del descenso de gradiente manual

El método del **descenso de gradiente** (o *gradient descent*) es el algoritmo de búsqueda local de primer orden más elemental. Para resolver el problema de minimización sin restricciones:

$$\min_{x \in \mathbb{R}^n} f(x)$$

el algoritmo genera una secuencia de puntos $\{x^{(k)}\}$ partiendo de una estimación inicial $x^{(0)}$. Dado que el gradiente $\nabla f(x)$ apunta en la dirección de máximo crecimiento local, nos movemos en la dirección opuesta $-\nabla f(x)$ para disminuir el valor de la función:

$$x^{(k+1)} = x^{(k)} - \alpha \nabla f(x^{(k)})$$

donde:

- $x^{(k)}$ es el punto en la iteración k .

- $\alpha > 0$ es la **tasa de aprendizaje** (*learning rate*) o tamaño de paso.
 - Si α es demasiado pequeño, la convergencia será extremadamente lenta.
 - Si α es demasiado grande, el algoritmo puede oscilar e incluso divergir.
- $\nabla f(x^{(k)})$ es el gradiente de la función en el punto actual.

2.1.1 Criterio de parada

En una implementación real, la iteración se detiene cuando se cumple alguna de las siguientes condiciones: 1. Se alcanza un número máximo de iteraciones ($k \geq K_{\max}$). 2. El gradiente es prácticamente cero, lo que indica que estamos en un punto estacionario: $\|\nabla f(x^{(k)})\|_2 < \epsilon$, donde $\epsilon > 0$ es una tolerancia (por ejemplo, 10^{-6}).

2.2 Optimización de caja negra con `scipy.optimize.minimize`

El módulo `scipy.optimize` es el estándar en Python para optimización numérica general. Su función estrella es `minimize`, que actúa como interfaz común para diversos métodos numéricos.

2.2.1 Optimización sin restricciones: el método BFGS

El algoritmo **BFGS** (Broyden-Fletcher-Goldfarb-Shanno) es un método de tipo **Quasi-Newton**. A diferencia del método de Newton clásico, que requiere calcular la matriz hessiana inversa ($H^{-1}(x)$) en cada iteración (lo cual es costoso computacionalmente), BFGS aproxima dicha matriz de forma iterativa utilizando únicamente información de los gradientes sucesivos.

La llamada básica a `minimize` tiene el siguiente aspecto:

```
from scipy.optimize import minimize

res = minimize(fun, x0, method='BFGS', jac=grad_fun)
```

Donde:

- `fun`: Función objetivo que recibe un array de NumPy y devuelve un escalar.
- `x0`: Punto inicial (vector o lista de Python).
- `method='BFGS'`: Especifica que usaremos el algoritmo Quasi-Newton BFGS.
- `jac`: Función que calcula el gradiente (Jacobiano).

- **Importante:** Si `jac=None` (o no se proporciona), `minimize` estimará el gradiente numéricamente mediante **diferencias finitas**. Esto requiere evaluar la función objetivo múltiples veces por cada paso (al menos $n + 1$ evaluaciones, donde n es la dimensión), lo que incrementa el coste computacional y puede introducir imprecisiones numéricas.

El resultado devuelto (`res`) es un objeto `OptimizeResult` con los siguientes campos clave:

- `res.x`: El punto óptimo encontrado.
- `res.fun`: El valor de la función en el óptimo.
- `res.success`: Booleano que indica si el solver convergió con éxito.
- `res.message`: Mensaje descriptivo de la terminación.
- `res.nfev`: Número de evaluaciones de la función objetivo.
- `res.njev`: Número de evaluaciones del gradiente (Jacobiano).

2.2.2 Optimización con restricciones: el método SLSQP

Cuando existen restricciones analíticas, el método por defecto de SciPy es **SLSQP** (Sequential Least Squares Programming). SLSQP formula y resuelve aproximaciones cuadráticas del problema original sujeto a la linealización de las restricciones en cada paso.

Para definir restricciones en SciPy se utilizan diccionarios dentro de una lista: 1. **Inecuaciones:** En SciPy se asume que las restricciones de desigualdad están formuladas como $g(x) \geq 0$. Si tu problema teórico tiene la forma estándar $h(x) \leq c$, debes reescribirla como $c - h(x) \geq 0$. 2. **Ecuaciones:** Restricciones de igualdad de la forma $e(x) = 0$.

Ejemplo de sintaxis:

```
# Restricción:  $x_1 + 2x_2 \geq 4 \implies x_1 + 2x_2 - 4 \geq 0$ 
def restriccion_lineal(x):
    return x[0] + 2*x[1] - 4

# Lista de restricciones
restricciones = [
    {'type': 'ineq', 'fun': restriccion_lineal}
]

# Límites de no negatividad:  $x_1 \geq 0, x_2 \geq 0$ 
cotas = [(0, None), (0, None)]

res = minimize(fun, x0, method='SLSQP', bounds=cotas, constraints=restricciones)
```

2.3 Optimización Convexa Disciplinada con CVXPY

CVXPY es una biblioteca de modelado declarativo en Python diseñada específicamente para **optimización convexa**. A diferencia de SciPy (que trata el problema como una “caja negra” numérica), CVXPY analiza la estructura matemática del problema y comprueba si cumple las reglas de la **Programación Convexa Disciplinada (DCP)**.

2.3.1 Reglas DCP Básicas

Para que CVXPY acepte y resuelva un problema, este debe cumplir una serie de reglas de composición geométrica que aseguran que el problema es verdaderamente convexo:

- **Función Objetivo:** Si minimizamos, el objetivo debe ser convexo (p. ej., x^2 , $\|x\|_2$, $\sum x^2$). Si maximizamos, debe ser cóncavo (p. ej., $\sqrt{x}$, $\log x$).
- **Restricciones de desigualdad:** Deben tener la forma `convexo <= concavo` o `concavo >= convexo`.
- **Restricciones de igualdad:** Deben ser afines (lineales) de la forma `lineal == lineal`.
- **Operaciones prohibidas:** No se permite multiplicar variables entre sí (p. ej. `x * y` o `x[0] * x[1]`), ya que esto rompe las reglas de convexidad generalizadas de DCP. Para representar términos cuadráticos como $x^T P x$, se deben usar funciones específicas como `cp.quad_form(x, P)` o `cp.sum_squares(x)`.

2.3.2 Obtención de Soluciones Duales (Multiplicadores de Lagrange)

En la teoría de la optimización, las **variables duales** (o multiplicadores de Lagrange/KKT asociados a las restricciones) nos indican la sensibilidad del valor óptimo de la función objetivo ante pequeñas variaciones en los límites de las restricciones (también llamados *precios sombra*).

CVXPY recupera de forma nativa e inmediata estas variables duales:

```
import cvxpy as cp

# 1. Definir variables
x = cp.Variable(2, nonneg=True) # nonneg=True añade la restricción x >= 0

# 2. Definir objetivo
objetivo = cp.Minimize(0.5 * cp.sum_squares(x))

# 3. Definir la restricción y asignarle un objeto
```

```

restriccion = x[0] + 2*x[1] >= 4
restricciones = [restriccion]

# 4. Resolver
problema = cp.Problem(objetivo, restricciones)
problema.solve()

# 5. Obtener los multiplicadores de Lagrange (dualidad)
valor_dual = restriccion.dual_value
print("Multiplicador de Lagrange de la restricción:", valor_dual)

```

2.4 Comparativa: SciPy vs CVXPY

Característica	scipy.optimize.minimize	CVXPY
Paradigma	Numérico procedural (caja negra).	Declarativo (lenguaje de modelado algebraico).
Garantías	Encuentra óptimos locales en problemas no convexos.	Garantiza óptimos globales en problemas convexos.
Validación	No comprueba si el problema es resoluble o convexo.	Valida estrictamente bajo las reglas DCP.
Derivadas	Requiere el gradiente de forma explícita o diferencias finitas.	Realiza diferenciación simbólica/automática interna.
Acceso a Dualidad	Complejo y dependiente del solver interno.	Directo mediante la propiedad <code>.dual_value</code> .

3 Ejercicios de laboratorio propuestos

3.1 Ejercicio 1: descenso de gradiente manual

Implementa un script en Python para resolver mediante descenso de gradiente manual la minimización de la función cuadrática:

$$f(x, y) = x^2 + 2y^2$$

3.1.1 Tareas:

1. Define la función objetivo $f(x, y)$ y su gradiente analítico $\nabla f(x, y) = (2x, 4y)^T$ en Python.
2. Implementa el bucle iterativo partiendo de $x^{(0)} = (2, 2)^T$ con una tasa de aprendizaje constante $\alpha = 0.1$ durante un total de $K = 10$ iteraciones.
3. Imprime en cada iteración: el número de iteración, el punto $(x^{(k)}, y^{(k)})$ y el valor de la función objetivo $f(x^{(k)}, y^{(k)})$.
4. Reflexiona: ¿El algoritmo ha llegado al mínimo global exacto $(0, 0)$ en 10 iteraciones? ¿Por qué?

```
# Tu código aquí:
import numpy as np

def f(x):
    # x es un array de dimensión 2
    return x[0]**2 + 2*x[1]**2

def grad_f(x):
    return np.array([2*x[0], 4*x[1]])

# Punto inicial
x_k = np.array([2.0, 2.0])
alpha = 0.1
max_iter = 10

# Bucle de optimización
# ...
```

3.2 Ejercicio 2: optimización sin restricciones con SciPy

Resuelve la misma optimización sin restricciones del Ejercicio 1:

$$\min \quad f(x, y) = x^2 + 2y^2$$

utilizando la función `minimize` de `scipy.optimize`.

3.2.1 Tareas:

1. Realiza la optimización de dos maneras distintas:
 - **Caso A:** Sin proporcionar el gradiente (dejando que SciPy calcule las diferencias finitas).
 - **Caso B:** Proporcionando el gradiente analítico a través del parámetro `jac`.
2. Para ambos casos, utiliza el método 'BFGS' e inicializa en $x^{(0)} = (2, 2)^T$.
3. Imprime el informe completo de optimización de ambos casos. Compare los valores finales de `nfev` (evaluaciones de la función) y `njev` (evaluaciones del gradiente) de cada caso y con los de tu método manual. ¿Qué diferencias observas?

```
# Tu código aquí:
from scipy.optimize import minimize

# Caso A: sin gradiente
# ...

# Caso B: con gradiente analítico
# ...
```

3.3 Ejercicio 3: optimización restringida con CVXPY y SciPy (Teoría y Práctica)

Consideremos el siguiente problema de programación cuadrática con restricciones lineales y de no negatividad:

$$\begin{array}{ll} \underset{x_1, x_2}{\text{minimizar}} & f(x_1, x_2) = \frac{1}{2}(x_1^2 + x_2^2) \\ \text{sujeto a} & x_1 + 2x_2 \geq 4 \\ & x_1 \geq 0, \ x_2 \geq 0 \end{array}$$

3.3.1 Tareas teóricas:

1. Escribe el Lagrangiano del problema utilizando los multiplicadores de Lagrange μ (para la restricción principal) y λ_1, λ_2 (para las de no negatividad).
2. Plantea las condiciones de optimalidad de Karush-Kuhn-Tucker (KKT).
3. Resuelve analíticamente el sistema KKT para obtener la solución óptima primal (x_1^*, x_2^*) y el multiplicador de Lagrange óptimo de la restricción principal (μ^*) .

3.3.2 Tareas de código:

1. Resuelve el problema utilizando **CVXPY**. Recupera el punto óptimo primal, el valor objetivo y el multiplicador de Lagrange asociado a la restricción $x_1 + 2x_2 \geq 4$ empleando su propiedad `.dual_value`.
2. Resuelve el problema utilizando **scipy.optimize.minimize** con el método 'SLSQP'. Define correctamente las cotas de no negatividad (**bounds**) y la restricción principal como un diccionario (recuerda que SciPy asume que las restricciones de desigualdad son de la forma $g(x) \geq 0$).
3. Compara los resultados de ambos códigos con tu cálculo analítico. ¿Coinciden el punto óptimo y el multiplicador de Lagrange obtenido? Explica los resultados.

```
# Tu código aquí:

# 1. Resolución con CVXPY
# ...

# 2. Resolución con SciPy SLSQP
# ...
```